

Praline

Explain and test helper-aware C loop parallelization

SegFault P05

Local prototype / C11 / Clang / OpenMP

The problem

```
output[i] = apply(input[i]);
```

A helper can perform independent arithmetic.

A similar helper can increment shared state.

The decision requires effects and dependencies.

How it works

Clang syntax tree and declaration identities
Helper effects and loop dependency checks
Explicit alias and bounds assumptions
Separate OpenMP source
Whole-output validation and repeated timings

Seeing through a call

Helper map: safe
INDEPENDENT_MAP

Global-write helper: unsafe
GLOBAL_WRITE: kernel loop / apply / counter write
Restrict contracts and separate allocations are explicit.

Generated code

CPU: generated, compiled and output-validated

```
#pragma omp parallel for default(none) shared(input, n, output)
```

GPU: generated and host syntax checked, device unverified

```
map(to: input[0:n]) map(tofrom: output[0:n])
```

Helper apply is made device-available with declare target.

Results

n=1024: 0.0010 ms serial / 0.2780 ms CPU

n=65536: 0.0160 ms serial / 0.2950 ms CPU

n=1048576: 0.2730 ms serial / 0.5450 ms CPU

Whole-process speedup at n=1048576: 0.780x

Whole-output cases passed: 30

5 trials after 1 warmup per variant. Kernel medians above.

Apple Clang 21 / arm64 Mac / libomp 23.1.1

Delivery and gaps

CLI, deterministic examples and execution harness

Plain HTML report with raw evidence

Focused C11 subset with explicit assumptions

GPU hardware verification remains unavailable

Demo recording and external publication remain pending